Project O - Phase 3: Semantic Analysis and AST Optimization

Compiler Construction, Innopolis University · Kotlin/JVM 21

1. Introduction

Phase 3 extends the existing O compiler with semantic validation and four AST optimization passes. The earlier hand-written lexer, recursive-descent parser, source positions, and tree printer remain the foundation. The pipeline is source -> tokens -> original AST -> semantic diagnostics -> optimized AST. Optimization requires successful validation. The default command still prints the parser tree, while explicit flags enable analysis and before/after output. This phase ends at AST transformation; it does not execute O programs or produce JVM bytecode.

The repository audit compared the language PDF, documentation, implementation, tests, and examples before changes. The original suite passed all 104 tests: 51 lexer tests and 53 parser tests. O variables infer their types from mandatory initializers, classes define nominal types, and operators are ordinary method calls. Consequently, implementing conventional binary-expression folding would have missed the language's actual AST.

2. Semantic analysis architecture

`SemanticAnalyzer.analyze(program)` creates fresh analysis state and returns a `SemanticResult` containing diagnostics and a separate `SemanticModel`. The model associates expression identities with resolved `OType` values and keeps the inheritance relation for assignability queries. Analysis never rewrites the immutable parser nodes. `SemanticDiagnostic` carries a message and one-based line/column; `requireSuccess()` exposes failures through `SemanticException` for independently tested optimizer entry points.

Analysis first registers predefined and user classes, validates the inheritance graph, collects fields and routine signatures, and checks overrides. It then infers field types and visits method and constructor bodies. Each lexical `Scope` has a local binding table and a parent; lookup proceeds outward. Initializers are checked before local bindings are inserted. Duplicate names within one scope fail, while nested branches and loops may shadow outer names. Field initialization checks prevent direct reads of not-yet-initialized fields.

Class and method collection resolves parser ambiguities without capitalization heuristics. `Counter()` and `calculate()` share the syntax `Call(Name, args)`; the symbol tables determine construction versus invocation. Both explicit zero-argument calls and bare zero-argument member accesses are supported. Collected signatures allow mutual references and match forward headers with their implementations. Unresolved, repeated, or conflicting declarations are diagnosed rather than treated as callable placeholders.

3. Implemented semantic checks

Declaration validation detects unknown variables, assignment targets, classes, types, parents, and methods. Duplicate classes, fields, locals, parameters, method signatures, and constructor signatures are rejected. Inheritance uses cycle detection, and inherited member lookup follows the validated ancestry. Overrides preserve parameter signatures and require a compatible return type.

Type checking covers inferred initialization, assignment, Boolean conditions, method/constructor arguments, and returned values. Assignability is nominal and transitive: a derived instance can replace a base-typed value. Integer and Real are distinct types, so assignment does not silently convert between them. The library's numeric constructors, conversions, and mixed arithmetic overloads provide the explicitly supported alternatives. Array/List uses retain their element type, with invariant collection arguments and checked library methods.

Overload resolution compares argument counts and assignability, then requires a unique most-specific applicable signature. It reports absent or ambiguous matches instead of selecting the first method with a matching name. Value methods require compatible returns and structurally returning paths; both branches of a nonconstant conditional must return unless a later statement does. Loops are not assumed to return. Void methods and constructors cannot return a value, and a void expression cannot initialize a variable.

For example, the statement `counter := 1` in `semantic_undefined_variable.o` reports `Undefined variable 'counter'` at 4:9. In `semantic_type_error.o`, `var counter : 0` followed by `counter := true` reports `Cannot assign Boolean to variable 'counter' of type Integer` at 5:9.

4. Implemented AST optimizations

`Optimizer` reconstructs immutable AST nodes and retains meaningful source positions. Its public individual passes validate input, and the combined API runs selected passes in a fixed order until structural equality establishes a fixed point. The original AST remains available for comparison.

Constant folding traverses children before their parents and recognizes supported built-in calls on literals. Thus `2.Plus(3.Mult(4))` becomes `14`, `3.Less(5)` becomes `true`, and `true.And(false).Not()` becomes `true`. Integer arithmetic, numeric comparisons, Boolean operations, selected unary operations/conversions, and conservatively safe Real operations are supported. User-defined methods and constructors are never evaluated. Overflow, division or remainder by zero, nonfinite results, and unsafe Real rounding cases retain their calls. For example, `1.Div(0)` and `0.1.Plus(0.2)` are preserved.

Constant conditional simplification selects a statically known Boolean branch. A branch containing no top-level local declaration can replace the conditional directly. Otherwise the pass retains an `If true` wrapper around the selected statements and removes the alternative. This preserves lexical scope without changing the grammar or introducing an artificial block node. The demonstration deliberately shadows a branch-local name after the conditional, making scope preservation visible.

Unreachable code elimination scans each statement list after recursively processing nested bodies. A return ends the list; an if/else whose alternatives both return does likewise. In `optimization_unreachable.o`, `return 42` followed by `var unused : 5` loses the declaration. The pass makes no assumption that an arbitrary loop terminates or returns.

Unused variable elimination resolves references to individual lexical bindings and removes only unreferenced local declarations with literal initializers. Assignment references also keep a binding alive. Fields are never removed. The example deletes `var removable : 7` while preserving an actually read local, an arbitrary method invocation, object construction, and a division-by-zero initializer. This conservative subset avoids needing a whole-program effect or exception analysis.

5. Testing and demonstration

The clean command `./gradlew clean build installDist` passed. The complete suite passed **278 tests, with 0 failures, errors, or skipped tests**: 51 lexer, 53 parser, 114 semantic, 6 scope, 40 optimizer, and 14 CLI tests. Phase 3 adds 174 tests without changing the original 104. Focused semantic/scope, optimizer, and CLI runs also passed.

The automated tests extend the unmodified lexer/parser suite with semantic, scope, type, optimizer, and CLI checks. Coverage includes valid overloads and inheritance, bad symbols and signatures, return paths, shadowing, unchanged analysis input, nested transformations, required non-transformations, source positions, and optimizer idempotence. A small test-only interpreter additionally compares selected return values and side-effect counters before and after optimization; it is not a production O runtime. The repository contains a process verifier, `scripts/verify_phase3.py`, which checks the installed executable's exit codes and output across working examples, invalid inputs, and selected passes. All **49 installed-CLI process checks passed**.

Build and reproduce with `./gradlew clean build installDist`, `./gradlew test`, and `python3 scripts/verify_phase3.py`. Analyze using `build/install/o-compiler/bin/o-compiler --analyze examples/semantic_valid.o`. For a visible transformation, use `--optimize --passes=fold --ast-before --ast-after examples/optimization_constant_folding.o` with the same launcher. The full lab sequence is in `docs/PHASE3_DEMO.md`. Semantic failures return 1 and prevent optimization; invalid CLI usage returns 2.

6. Limitations and conclusion

The implementation provides multiple non-mutating checks and four actual AST transformations while preserving the earlier parser interface. General user-defined generics, static array-bounds proofs, inlining, constant propagation, and whole-program effects are outside this phase. Numeric and scope safeguards deliberately leave some expressions unoptimized. No runtime equivalence claim is inferred beyond the tested subset. Field-initializer order checks cover direct reads; dependencies hidden in arbitrary method calls are not proven. The result is a testable semantic boundary and a conservative optimization pipeline ready for a future code-generation phase.